Akhilesh Singh

Rewa, India singhakhilesh19468@gmail.com
linkedin.com/in/akhilesh-singh-dev github.com/Akhilesh-Singh-0 x.com/singh_akhil2272

Technical Skills

Languages	TypeScript, JavaScript, SQL
Backend	Node.js, Express.js, Fastify, WebSockets, Redis (Pub/Sub, atomic counters), BullMQ, OpenTelemetry (OTLP), REST API Design, Zod
Databases	PostgreSQL, MongoDB, Prisma ORM
Auth / Access	Clerk, JWT, RBAC, API Key Auth
Frontend	Next.js 15, React 19, TanStack Query, Tailwind CSS, shadcn/ui
Infra	Docker, Turborepo, Railway, Vercel, Neon
Concepts	Real-time Systems, Observability, Event-driven Architecture, Async Job Processing, Idempotent Processing, Multi-tenant Architecture

Projects

Beacon — Real-time Execution Graph & Debugger for AI Agents

Live GitHub

Node.js · TypeScript · Fastify · PostgreSQL · Redis · BullMQ · OpenTelemetry · WebSockets · Next.js · Prisma · Turborepo

- Built an **OpenTelemetry (OTLP) ingestion API** that authenticates by workspace API key, validates payloads and enqueues spans via BullMQ — returns `202 Accepted` without waiting on any database work.
- **Load tested the production ingestion endpoint at ~378 req/s with 100 concurrent clients, 0% request failures**, p50 latency 224ms on Railway.
- Designed a **span worker that reconstructs each agent execution as a graph** of runs, nodes and edges from `parentSpanId`; token usage is extracted from span attributes and stored per node.
- Made processing **idempotent** via a unique constraint on `spanId` with silent `P2002` handling — queue retries and duplicate OTLP deliveries never create duplicate nodes or edges.
- Streamed live execution events to the dashboard over **run-scoped WebSockets backed by Redis Pub/Sub** — dedicated subscriber connection, deduplicated subscriptions, cleanup when the last client leaves.
- Implemented **stuck-node detection** with BullMQ repeatable jobs instead of `setInterval`, plus graceful shutdown that drains workers and queues cleanly on deploy.

Guardrail — AI API Cost Enforcement Proxy

Live GitHub

Node.js · TypeScript · PostgreSQL · Redis · BullMQ · WebSockets · Next.js 15 · Clerk · Turborepo

- Built a **reverse proxy that enforces per-user AI spend limits before the provider is called** — 402 returned instantly on over-limit requests; provider never reached, charge never incurred.
- Eliminated double-charging under concurrent load using a **Redis Lua script** that atomically checks and increments spend in one transaction — naive read-check-increment allows two requests to both pass a $5 limit and charge $5.06; Lua closes the race window entirely.
- Decoupled the hot path from side effects via **BullMQ** — response returns to client immediately; background worker writes UsageEvent to Postgres, publishes to Redis Pub/Sub, WebSocket pushes spend update to browser in ~100ms.
- Implemented **cache stampede protection** on limit lookups via Redis lock — prevents thundering herd on cache miss; rebuilt Redis counters from Postgres on cold start to survive restarts without data loss.
- Stored all costs as **micro-units ($\times 10^6$ integers)** with Decimal.js arithmetic — floats never touch billing; deduplication via Postgres unique constraint on `idempotencyKey` survives Redis restarts with no extra infra.
- **Load tested at ~134 req/s, 50 concurrent connections, zero errors** on Railway free tier; anomaly detection fires threshold alerts at 80% and 100% spend against a 7-day rolling baseline per user.

Education

Bachelor of Technology — Mechanical Engineering 2022 – 2026
Madhav Institute of Technology & Science, Gwalior

Certifications

- **Meta Back-End Developer Professional Certificate** — Coursera 2026
- **PostgreSQL for Everybody Specialization** — University of Michigan, Coursera 2026